

HOSPITAL MANAGEMENT SYSTEM

Process Documentation

Document Type	Process Documentation
Version	1.0
Date	04 May 2026
Language	Java (JDK 17+)
Database	MySQL 8.x
Status	Final

1. Introduction

The Hospital Management System (HMS) is a Java-based, console-driven application that centralises the operational workflows of a hospital into a single, database-backed platform. It replaces paper-based and siloed processes by providing a unified interface for patient registration, clinical scheduling, bed management, prescriptions, billing, and human resources.

1.1 Scope

The system covers the following functional areas:

- Patient lifecycle — registration through discharge
- Doctor and appointment management
- Ward and bed allocation
- Prescription and pharmacy workflow
- Billing and payment tracking
- Staff and HR records
- Live dashboard reporting

1.2 Objectives

- Reduce manual data-entry errors through structured database storage
- Provide real-time visibility into bed occupancy and daily appointments
- Enforce business rules such as appointment slot conflict detection
- Maintain complete audit trails for billing and clinical records
- Deliver a single-file deployment that requires no application server

2. System Architecture

The HMS follows a layered architecture pattern implemented within a single Java compilation unit. Each concern is separated into clearly named inner classes and static methods, enabling maintainability without the overhead of a multi-module build system.

Layer	Responsibility	Key Classes / Methods
Presentation	Console menus, user prompts, formatted output	*Menu() methods, toString() overrides
Service	Business rules, validation, orchestration	bookAppointment(), recordPayment(), admitPatient()
Repository (DAO)	SQL execution via JDBC PreparedStatement	All DB query methods per entity
Domain Model	Plain Java objects representing entities	Patient, Doctor, Appointment, Bed, Invoice, Staff
Infrastructure	DB connection singleton, schema init	getConnection(), initDatabase()

2.1 Design Patterns Used

Pattern	Where Applied	Benefit
Singleton	DatabaseConnection.getConnection()	Single shared JDBC connection across all operations
Repository / DAO	Per-entity DB methods	Decouples SQL from business logic
Service Layer	Patient/Doctor/Appointment services	Centralises validation and orchestration
Template Method	Menu dispatch (switch/case)	Consistent navigation structure
Value Object	Domain model inner classes	Immutable data containers with toString()

3. Core Process Flows

3.1 Patient Registration Process

The patient registration process captures demographic and medical baseline data and persists it in the patients table. The flow is strictly sequential with no branching except for optional fields.

Step	Actor	Action	System Response
1	Receptionist	Select option 1 — Patients → Register	Display registration form prompt
2	Receptionist	Enter full name, phone, email	Buffer input for batch INSERT
3	Receptionist	Enter blood group, address, DOB	Validate date format YYYY-MM-DD
4	System	Execute INSERT INTO patients	Return auto-generated patient_id
5	System	Display confirmation with new ID	Record is now queryable

Business Rule

Phone numbers must be unique (UNIQUE constraint on the patients table). Duplicate phone entry triggers a SQLException which is caught and reported to the user.

3.2 Appointment Booking Process

Appointment booking enforces a slot-conflict check before committing the record. A doctor cannot have two SCHEDULED appointments at the same date and time.

Step	Actor	Action	System Response
1	Receptionist	Select Appointments → Book	Display patient and doctor lists
2	Receptionist	Choose patient ID and doctor ID	Validate IDs exist via FK constraints
3	Receptionist	Enter date/time (YYYY-MM-DD HH:MM)	System queries for slot conflicts
4	System	COUNT(*) conflict check	If count > 0: reject with message
5	System	Execute INSERT INTO appointments	Return appointment_id
6	System	Confirm booking to user	Status defaults to SCHEDULED

Business Rule

Status transitions: SCHEDULED → COMPLETED or SCHEDULED → CANCELLED. Only these transitions are valid. Status is stored as a MySQL ENUM.

3.3 Bed Admission & Discharge Process

Step	Actor	Action	System Response
1	Nurse	Select Ward & Beds → Admit patient	Display available beds and patient list
2	Nurse	Choose bed ID and patient ID	System validates bed.occupied = FALSE

Step	Actor	Action	System Response
3	System	UPDATE beds SET occupied=TRUE, patient_id=?	If bed already occupied: error returned
4	Nurse	Select Discharge when ready	Enter bed ID to release
5	System	UPDATE beds SET occupied=FALSE, patient_id=NULL	Bed returns to available pool

3.4 Billing & Payment Process

The billing process supports partial payments. An invoice can receive multiple payments over time; the system recalculates status automatically after each payment.

Step	Actor	Action	System Response
1	Billing Clerk	Select Billing → Create Invoice	Display patient list
2	Billing Clerk	Select patient, add line items	Each item inserted into invoice_items
3	System	Sum line items, UPDATE invoices.total_amount	Invoice created with status PENDING
4	Billing Clerk	Select Record Payment, enter invoice ID	System fetches current paid_amount
5	System	Add new payment to paid_amount	If paid >= total: status = PAID
6	System	If paid < total: status = PARTIAL	Balance is tracked for follow-up

3.5 Prescription Process

Step	Actor	Action	System Response
1	Doctor	Select Prescriptions → Issue	Display patient and doctor lists
2	Doctor	Select patient, doctor, optional appointment	Optional FK to appointments table
3	Doctor	Enter medication, dosage, instructions	Data buffered for INSERT
4	System	INSERT INTO prescriptions	Return prescription_id
5	Pharmacist	Select Prescriptions by Patient	Full history displayed by date desc

4. Database Design

The schema consists of eight tables, automatically created at application startup via `initDatabase()`. Foreign key relationships enforce referential integrity. All tables use `AUTO_INCREMENT` primary keys.

Table	Primary Key	Foreign Keys	Key Columns
patients	patient_id	—	name, phone (UNIQUE), blood_group, dob
doctors	doctor_id	—	name, specialization, consultation_fee
appointments	appointment_id	patient_id, doctor_id	appt_datetime, status (ENUM)
beds	bed_id	patient_id (nullable)	ward, type, occupied (BOOL)
prescriptions	prescription_id	patient_id, doctor_id, appointment_id	medication, dosage, prescribed_date
invoices	invoice_id	patient_id	total_amount, paid_amount, status (ENUM)
invoice_items	item_id	invoice_id	description, amount
staff	staff_id	—	name, role, department, salary

4.1 ENUM Types

Two tables use MySQL ENUM columns to enforce allowed values:

- `appointments.status` — values: `SCHEDULED`, `COMPLETED`, `CANCELLED`
- `invoices.status` — values: `PENDING`, `PAID`, `PARTIAL`

4.2 Connection Management

Singleton Pattern	<code>getConnection()</code> implements a double-checked locking singleton. The connection is reused across all operations in a session. On exit (option 0), <code>conn.close()</code> is called explicitly to release resources.
-------------------	---

5. Error Handling & Validation

All database operations are wrapped in try-with-resources blocks ensuring PreparedStatement and ResultSet objects are always closed. The main menu loop catches both SQLException and NumberFormatException to prevent crashes from invalid user input.

Error Type	Where Caught	User Feedback
ClassNotFoundException	main() — driver loading	Driver not found message; application exits cleanly
SQLException	initDatabase(), all CRUD methods	Error message printed; menu loop continues
NumberFormatException	main() menu loop	Invalid number message; user can retry
Duplicate phone (SQL)	registerPatient()	SQLException caught — duplicate entry message shown
Slot conflict	bookAppointment()	Pre-query check; rejection message before any INSERT
Bed already occupied	admitPatient()	UPDATE affects 0 rows; specific message returned

6. Deployment & Configuration

6.1 Prerequisites

Component	Version	Notes
Java JDK	17 or later	Text blocks (triple-quote strings) require JDK 15+
MySQL Server	8.0 or later	Must be running and accessible on configured host
MySQL JDBC Driver	8.x (mysql-connector-j)	Must be on classpath at compile and runtime

6.2 Configuration

Edit the three constants at the top of HospitalManagementSystem.java:

```
DB_URL = "jdbc:mysql://localhost:3306/hospital_db?createDatabaseIfNotExist=true&useSSL;=false&allowPublicKeyRetrieval;=true"
DB_USER = "root" // your MySQL username
DB_PASS = "password" // your MySQL password
```

6.3 Build & Run Commands

```
# Compile
javac -cp mysql-connector-j-8.x.x.jar HospitalManagementSystem.java

# Run (Linux / macOS)
java -cp .:mysql-connector-j-8.x.x.jar HospitalManagementSystem

# Run (Windows)
java -cp .;mysql-connector-j-8.x.x.jar HospitalManagementSystem
```

First Run	On first launch the system automatically creates the hospital_db database and all 8 tables if they do not exist. No manual SQL setup is required.
-----------	---

7. Testing Checklist

The following test scenarios should be executed to verify correct system behaviour before go-live.

#	Test Scenario	Expected Result	Status
1	Register patient with valid data	New patient_id returned; record in DB	■
2	Register patient with duplicate phone	Error message; no duplicate inserted	■
3	Book appointment — available slot	appointment_id returned; status SCHEDULED	■
4	Book appointment — conflicting slot	Rejection message; no record inserted	■
5	Admit patient to available bed	Bed occupied=TRUE; patient linked	■
6	Admit patient to occupied bed	Error message; original occupant unchanged	■
7	Discharge patient from bed	occupied=FALSE; patient_id=NULL	■
8	Create invoice with 3 line items	Total equals sum of items; status PENDING	■
9	Record partial payment	paid_amount updated; status PARTIAL	■
10	Record final payment	paid_amount = total_amount; status PAID	■
11	List today's appointments	Only today's records shown	■
12	Dashboard summary counts	All counts match actual DB row counts	■